There are other ways to structure this (perhaps without a paddle class), but it's important to see how the program is broken up into small composable units even though it's handling multiple users. Regardless of the source of events, this technique helps you to manage event streams. With first-class functions and a terse syntax for both functions and objects, CoffeeScript makes it possible to write readable code.

### 9.4.5. Multiple event sources

That works for a single source of events, but what happens when you have multiple sources of events? Suppose you have two sources of data:

```
source1 = [13,14,15]
source2 = [23,24,25]
```

For arrays, to `zip` them means to create a new array by interleaving the two arrays together, like a zipper:

```
zip = (a, b) ->
  zipped = []
  while a.length or b.length
    do ->
      fromA = a.pop()
      fromB = b.pop()
      if fromB then zipped.push fromB
      if fromB then zipped.push fromA
  zipped.reverse()

zip source1, source2
# [13,23,14,24,15,25]
```

Streams of events, when treated as arrays, can be zipped together in the same way. With two event streams where the source order doesn't matter, zipping them together as they arrive is done in the same way as two arrays are zipped together.

## 9.5. Summary